

Lecture 3: Basic Data Types and Functions

1

Definitions

2

Haskell Programming Language

Definitions

❖ Definitions

- A definition **associates** a **name/identifier** with a value of a particular **type** using **name::Type**
- the **::** means **“is a/an”**

age :: Integer

- associates the **name** **age** with an **Integer** type
- That is, age is an Integer

- Names/identifiers should **begin with a small letter**
- The **types** should **begin with a capital letter**
- Remember that in Haskell, **every value** and **every expression** has a **type**
- We can **check the type** of a **value or expression** using the **:type** command

3

Haskell Programming Language

Definitions

- A function takes **input** (s) and returns an **output** that depends on the inputs
- **Function declaration** consists of the **function name**, the **argument list** and the **output** all separated by the **->** sign
- **Function definition** is where you **actually define the function**
- **Example**

```
square :: Integer -> Integer --declare the function square
```

```
square x = x * x -- function definition
```

- the **->** indicates that the identifier **square** is a **function**
- the **input** to square function is an **Integer**, and the **output** is also an **Integer**

4

Haskell Programming Language

Definitions

```
addNums :: Integer -> Integer -> Integer  --declare the function addNums
addNums x y = x + y                        -- function definition
```

- The function is defined as `addNums`
- It takes `two arguments` of `Integer` type and `returns an Integer`

5

Data Types

7

Haskell Programming Language

Data Types

- In Haskell, every statement is considered as a mathematical expression that can be categorized by **type**
- A type is a collection of values grouped together because they are the same sort of thing, even though they are different
- Types can be divided into **two kinds**
 - **Basic types** whose values are given as **primitive** – example include numbers (the type **num**), truth-values the (the type **bool**) and characters (the type **char**)
 - **Compound** (or **derived**) types, whose values are constructed from those of others – examples include **lists** which contain several values of a single type(e.g. [Integer]), and **String** which list of characters ([Char])

9

Haskell Programming Language

Data Types

- There are two stages of analysis when an expression is submitted for evaluation
- ❖ **Syntax analysis**
 - The first stage
 - The expression is first checked to see whether it conforms to the correct syntax laid down for expressions
 - If it does not, the computer signals a **syntax error**
 - This stage is called **syntax-analysis**

10

Haskell Programming Language

Data Types

- There are two stages of analysis when an expression is submitted for evaluation
- ❖ **Type analysis**
 - Second stage
 - Performed when the expression conforms to the syntax
 - If the expression does, then the expression is analyzed to see if it possesses a sensible type
 - This stage is called *type-analysis*
 - If the expression fails to pass this stage, the computer signals a **type error**

11

Haskell Programming Language

Data Types

- There are two stages of analysis when an expression is submitted for evaluation
- Only if the expression passes both the syntax analysis and type analysis stages can the process of evaluation begin
- Similar remarks apply to definitions before a script is accepted
- **Strong typing** is important because adherence to the discipline can help in the design of clear and well-structured programs
- When declaring the variable, you must also specify the variable type
- What is more, a wide range of logical errors can be trapped by any computer which enforces it

12

Haskell Programming Language

Data Types

❖ **Integers:** `Int` and `Integer` type

- The Haskell `Int` and `Integer` contains whole numbers, positive numbers, negative numbers, and 0

e.g. 0
 76
 -890
 921425

- `Int` is the type of limited precision, generally of 2 bytes, and is bounded
- Haskell `Integer` type is not bounded, therefore can represent very large numbers
- Every whole number within the range of 2147483647 to -2147483647 comes under the `Int` type class
- `Integer` can be considered as a superset of `Int`, this value is not bounded by any number, hence an `Integer` can be of any length without any limitation

13

Haskell Programming Language

Data Types

- **Arithmetic** on integers is done using infix arithmetical operations: `-`, `*`, `/`, `+`, `-`, `div`, `mod`, `abs`, `negate`, `^`,

- `+` addition
- `-` subtraction
- `*` multiplication
- `/` division
- `^` exponentiation

- Addition, subtraction and multiplication are all left associative
- Multiplication and division have a higher precedence than addition and subtraction which have the same precedence as each other
- **Example** : `2+3+7*11 = ?` `2+3-7*11=?`

14

Haskell Programming Language

Data Types

❖ **Remember:**

- Haskell also has the unary prefix operator `-`
- It is used to indicate negative numbers
- Remember to enclose negative numbers in parenthesis if used after an arithmetic operator

15

Haskell Programming Language

Data Types

▪ **Haskell also has the following binary prefix operators:**

- `mod` – gives the remainder from a whole number division e.g., `mod 14 3 = 2`
- `div` – gives the whole number division e.g., `div 14 3 = 3`
- `negate` – change the sign of an integer e.g., `negate (-3) = 3`
- `abs` – the absolute value of an integer; remove the sign e.g., `abs (-2.56) = 2.56`
- `min` – returns the minimum of two numbers
- `max` – returns the maximum of two numbers
- `gcd` – greatest common denominator
- `lcm` – lowest common multiple

16

Haskell Programming Language

Data Types

- **Relational operators** take two integers and return a boolean True or False
- These are `>`, `<`, `>=`, `==`, `/=`, `<=`,
- Examples
 - `5 < 3` returns `False`
 - `5 > 3` returns `True`
 - `5 == 5` returns `True`

17

Haskell Programming Language

Data Types

❖ **The Booleans** : `Bool` type

- This type is used for comparisons
- In Haskell it is called `Bool` and contains only two values `T` or `F`
- The operators provided are

<code>&&</code>	<code>and</code>
<code> </code>	<code>or</code>
<code>not</code>	<code>not</code>

```
Prelude> True && True
True
Prelude> True && False
False
Prelude> True || True
True
Prelude> True || False
True
```

- Operators `||` and `&&` are binary infix, whereas `not` is a prefix unary
- Both `||` and `&&` are **right associative** and `&&` has a higher precedence than `||`
- `False && False && False || True = ?`
- `False || True && False && False = ?`

18

Haskell Programming Language

Data Types

❖ **The Booleans** : `Bool` type

- Booleans can be the **arguments** to, or the **results of functions**
- Booleans values can also be **compared for equality or inequality** using the `==` and `/=` operators
- The Boolean functions can be defined as follows:

○ `|| :: Bool -> Bool -> Bool`

○ `&& :: Bool -> Bool -> Bool`

○ `not :: Bool -> Bool`

19

Haskell Programming Language

Data Types

❖ **The Booleans** : `Bool` type

- **Fill in the truth tables below:**

a	not a
T	
F	

a	b	a b
T	T	
T	F	
F	T	
F	F	

a	b	a&& b
T	T	
T	F	
F	T	
F	F	

20

Haskell Programming Language

Data Types

❖ **Floating point numbers:** `Float` type

- Division of integers can give **non-integer result** , that is, **numbers with fractions**
- `Float` is a real floating point with **single precision**
- Haskell uses **fixed amount of space to store floats**, meaning that not all fractions can be represented as floating point numbers
- This can affect the **accuracy of floating-point numbers**
- **Scientific notation** representation of numbers is allowed in Haskell , which is **more convenient**
- E.g. $1.343e7 \quad 1.343 \times 10^7 = 1 \ 3430 \ 000$
 $-3.412e03 \quad -3.412 \times 10^3 = 3412$
 $567.8e-2 \quad 567.8 \times 10^{(-2)} = 5.678$

21

Haskell Programming Language

Data Types

❖ **Floating point numbers:** `Float` type

- Division of integers can give **non-integer result** , that is, **numbers with fractions**
- `Float` is a real floating point with **single precision**
- Haskell uses **fixed amount of space to store floats**, meaning that not all fractions can be represented as floating point numbers
- This can affect the **accuracy of floating-point numbers**
- **Scientific notation** representation of numbers is allowed in Haskell , which is **more convenient**
- E.g. $1.343e7 \quad 1.343 \times 10^7 = 1 \ 3430 \ 000$
 $-3.412e03 \quad -3.412 \times 10^3 = 3412$
 $567.8e-2 \quad 567.8 \times 10^{(-2)} = 5.678$

22

Haskell Programming Language

Data Types

❖ **Floating point numbers:** `Float` type

- A range of operators and functions are provided
- Basic arithmetic `+`, `-`, `*`, `/`, `^`, `**`
- Equality and ordering operations `==`, `/=`, `<`, `>`, `<=`, `>=`
- The unary prefix operator `-` for negative numbers can be used
- The constant `pi` is also defined

23

Haskell Programming Language

Data Types

- Haskell has some useful functions for converting floating-point numbers into limited-precision integers , that is `Float -> Int`
- `ceiling` – returns the closest integer greater than or equal to a given number
e.g., `ceiling 2.3 = 3`
- `round` – which rounds a floating-point number e.g., `round 2.7 = 3` `round 2.2 = 2`
- `floor` – returns the greatest integer not greater than the value represented e.g., `floor 2.3 = 2`

24

Haskell Programming Language

Data Types

- There are several useful **unary prefix operators available for floating point numbers**
 - `cos` cosine
 - `sin` sine
 - `tan` tangent
 - `acos` inverse cosine
 - `asin` inverse sine
 - `atan` inverse tangent
 - `log` natural logarithm
 - `exp` power of e
 - `sqrt` positive square root

25

Haskell Programming Language

Data Types

- ❖ **Double floating point:** `Double` type
 - `Double` is a real floating point with **double precision**
 - `Double` is a **good choice for real numbers** in the vast majority of cases
 - This is because the single-precision counterpart of `Double`, `float`, is **usually less attractive due to further loss of precision**

26

Haskell Programming Language

Data Types

❖ **Characters:** Char type

- These are **letters, digits, special characters** like space, tab, new line, end of file...
- Literal characters are **enclosed in single quotes** e.g., 'y', '6'
- The single opening quotation mark is simply written between single closing quotation marks, thus ''.
- Some special characters are :

`'\t'` tab

`'\n'` new line

`'\\'` backslash

`'\''` single quotes

`'\''''` double quotes

27

Haskell Programming Language

Data Types

❖ **Characters:** Char type

- Have standard ways of **coding characters as integers** such as ASCII, unicode
 - Capital letter A to Z have sequence code numbers 65 to 90
 - Small letters a to z have codes 97 to 122

e.g. `'\65'` and `'A'` have the same meaning
- Specify the operation:-
 - `getChar` which **gives the character** of an integer code
 - `getCode` which **gives the integer** code of a character

28

Haskell Programming Language

Data Types

- There are many **useful functions defined on characters** contained in the library `Data.Char`.
- To use these you have to import this module.
- These include
 - `toUpper`
 - `toLower`
- These are both of type `Char -> Char`
- The function `ord`, of type `Char -> Int`, turns a character into its ASCII code
- The function `chr`, of type `Int -> Char`, turns an ASCII code into a character
- Other useful functions are `:isAscii`, `isUpper`, `isLower`, `isAlpha`, `isDigit` and `isAlphaNum`.

29

Haskell Programming Language

Data Types

❖ **String type :**

- A **string** is nothing but a **collection of characters**
- Haskell follows the **conventional style of representing a string with double quotation**
- String is a **sequence of characters enclosed in double quotes** eg `"Tuesday"`, `"monkey \ndog \ncow"`
- **Empty string** has no characters
- Strings can be **joined using ++**
- e.g. `"cat"++"fish"` will print `catfish`
- How do we print the following?


```
cat
fish
```

30

Haskell Programming Language

Data Types

❖ **String type :**

- Some of the string operations:-
 - `length` which gives the **string length**
 - `cat` which **appends one string to the other string**
 - `substring`
- The `show` function converts from a value to a `String`
- The `read` function converts a string to a value

31

Haskell Programming Language

Data Types

❖ **Overloading**

- In Haskell, Integers, `Int` and Booleans can all be compared for equality using the `==` symbol
- The `==` symbol is used for equality over any type for which an equality operator can be defined
- This is called overloading
- Also the number 4 and 2 belong to both Integer and Float, they are overloaded

32

Haskell Programming Language

Data Types

❖ List type

- A list is a sequence of elements of the same type
- The elements enclosed in square brackets
- Lists in Haskell are homogeneous in nature, which means they won't allow you to declare a list of different kind of data type, that is any list like [1,2,3,4,5,a,b,c,d,e,f] will produce an error
- [T] for the type of all lists, whose elements are of type T
 - `[2,4,50] :: [Integer]`
 - `['2', '4', '2', 'r'] :: [Char]`
 - `["2","int","50"] :: [String]`
- length gives the number of elements in a list
 - An empty list [] is of length 0
 - The singleton list is of length 1

33

Haskell Programming Language

Data Types

❖ List type

- It is possible to have a list of lists such as:-
 - `[ ['2','s'], ['4','2','r'] ] :: [ [Char] ]`
 - `["2","int","50"] :: [String]`
- Some list functions are
 - length which gives the number of elements in a list
 - ++ which concatenates a list to the other list
 - occurs which gives the number of occurrences of a given element in a list

34

Haskell Programming Language

Data Types

❖ Tuple type

- A tuple is a **finite sequence of possibly different types**
- Components enclosed in **round brackets ()**
- A Tuple **can be considered as a List**, but there are differences in between a Tuple and a List
- A Tuple is an **immutable data type**, as we **cannot modify the number of elements at runtime**, whereas a **List is a mutable data type**
- List is a homogeneous data type, but Tuple is heterogeneous in nature, because a **Tuple may contain different type of data inside it.**
- Tuples are represented by single parenthesis(), lists by square brackets []

35

Haskell Programming Language

Data Types

❖ Tuple type

- The type of a tuple gives its **arity**
- Tuples must have a finite arity so that the tuple type can be calculated before evaluation
- No restrictions on the types of the components of a tuple, may include:-
 - Tuples of tuples
 - Tuples of lists
 - Lists of tuples
- Examples:-

```
(50, ("cs", False)) :: (Integer, (String, Bool))
```

```
(['2', 'a', 'x'], [False, False, False]) :: ([Char], [Bool])
```

```
[("cs", False), ("sc", True)] :: [(String, Bool)]
```

37

Functions

38

Haskell Programming Language

Functions

- A function **takes arguments and returns the results**
- Each argument has a type , results also
- Examples:-


```
add :: (Integer, Integer) -> Integer
isIn :: (String,[String]) -> Bool
add :: (String,[String]) -> [String]
head :: [Integer] -> Integer
fst :: (Integer, String) -> Integer
```
- Give a simple specification of :-
 - `tail` which gives the tail of list
 - `snd` which gives the second element of a pair

39

Introduction to Functional Programming

Functions

- A function **transforms input into output**
- Functions defined in a program are used to **write expressions to be evaluated**
- In the context of functional programming, a function can be thought of as **equivalent to a mathematical function**, i.e., **a black box mapping a given input to a unique output**



- The function gives an **output value which depends upon the input values(s)**
- The exact syntax of how to write a functions depends on the language that you are programming in

40

Introduction to Functional Programming

Functions

- Functions can make **code more efficient, easier to read, and elegant**
- A function can be called **and reused several times**
- Information can be **passed to a function** and the **function can send information back**
- Many programming languages have built-in functions that you can access in their library, but you can also create your own functions

41

Introduction to Functional Programming

Functions

❖ Void Functions

- A void function means that the function does not return a value
- Void functions are used to complete a set of instructions
- They are static and don't change and they always complete the same instructions
- e.g., the function **helloWorld** which outputs "Hello World"

42

Introduction to Functional Programming

Functions

❖ Functions That Require Values

- These are functions that need some data to be executed
- In the brackets, we can declare that data is needed to run our function
- Then we can use the data passed to our function in the function

❖ Functions That Return a Value

- These functions act on some data and return result
- This is particularly valuable when you want to alter data before you use it.
- For example, a function that requires two integers and will return the sum as an integer

43

Introduction to Functional Programming

Functions

- An important feature of functional programming is that functions are treated no differently from primitive values such as boolean values and integers
- A function
 - can be stored in a variable
 - can also be passed as an actual argument to another function
 - or even returned as a return value of another function
- Such values are called **first-class objects** because they are the most basic element comprising a program
- Hence functions are first-class objects in functional languages

44

Introduction to Functional Programming

Functions

- **Remember, FP functions are Pure functions**
- They always produce the same output for same arguments irrespective of anything else
- Also, they have no side-effects i.e. they do not modify any arguments or local/global variables or input/output streams
- Later property is called **immutability** – the pure function's only result is the value it returns
- This means that programs done using functional programming are easy to debug because pure functions have no side effects or hidden I/O
- Pure functions also make it easier to write parallel/concurrent applications

45

Introduction to Functional Programming

Functions

- To define a function

```
square :: Integer -> Integer
square n = n * n
```

- An example of a **definition**
- Associates a name/identifier with a value of a particular type

- The first line declares the function called `square`, taking an integer type to an integer type
- **Names/identifiers** should begin with a small letter and **types** with a capital letter
- The `::` means “is a/an”
- `n` is an unknown or a **variable**
- The **arrow** signifies that it is a function
- The **input**, of type integer, appears before the arrow
- The **output** is also of type integer, appearing after the arrow
- To evaluate the function `square` applied to an expression `y`, it becomes `square y = y * y`

46

Introduction to Functional Programming

Functions

- The variables in a function stand for arbitrary values
- These variables are called **formal parameters** of the function
- They stand for arbitrary values of the parameters
- The **actual parameters** are supplied when the function is applied
- Parameters are also called parameters
- A formal argument/parameter serves only as a placeholder for an actual parameter

47